

CORA Architecture Decision Records (ADRs)

These ADRs document key architectural decisions made for the FinServ Global **AI-Powered Regulatory Compliance Assistant**. Each ADR follows the standard format with context, the decision taken, alternatives considered, consequences, and the decision status. The decisions are based on the project's requirements for a **regulated financial environment** with strict compliance, security, and open-source constraints.

ADR 1: Choice of Vector Database (PostgreSQL + pgvector Extension)

Context: The Regulatory Compliance Assistant requires a vector database to store and query embeddings from regulatory documents. The vector store must handle thousands of regulatory text chunks, support similarity search for retrieval-augmented generation (RAG), and integrate with structured metadata (e.g., regulation type, version, jurisdiction) for filtering. Operating in a financial-services environment introduces additional constraints around security, data residency, and integration with existing systems. The solution must use **open-source components**, per project requirements, and ideally avoid introducing unnecessary infrastructure. Moreover, the system should scale and remain maintainable as the volume of regulatory documents grows.

Decision: Use **PostgreSQL with the pgvector extension** as the vector database for storing and querying text embeddings. All semantic embeddings of regulatory text (e.g., Basel III clauses, MiFID II guidelines, RBI circulars) are stored as vector columns in a PostgreSQL database, using pgvector to enable efficient approximate nearest-neighbor (ANN) queries. This unified approach allows vector embeddings and their metadata (document IDs, sections, version tags, etc.) to reside in a **single relational database**.

This decision leverages **PostgreSQL**, a proven production-grade relational database likely already familiar to the company's infrastructure, combined with **pgvector**, a robust open-source extension for vector similarity search. The combined solution meets our needs in terms of familiarity, ease of integration, and open-source compliance.

Alternatives Considered:

- *Dedicated Vector Database (e.g., Pinecone, Weaviate):* Specialized vector search engines offer high performance and advanced features (optimized ANN indexing, distributed scaling). However, they introduce **additional operational complexity**, requiring a separate cluster and new data pipelines. In a regulated environment, using a cloud-managed vector DB could pose compliance risks if not on approved infrastructure. Opted against this due to integration complexity and data governance concerns, especially for the initial prototype.
- *Custom In-Memory Store or Local Files:* Considered storing embeddings in memory or on the file system for simplicity. This approach avoids relying on any database, but it lacks persistence and complicates multi-node scaling and query filtering by metadata. It's not suitable for production-level reliability or traceability, so discarded this option.

Consequences:

- *Positive:* Using PostgreSQL + pgvector **simplifies our architecture** by allowing both structured data and vectors in one system. Avoid introducing a new technology that may not be approved for use in our enterprise environment. This choice lowers **operational overhead** – manage one database system instead of two. It’s open-source and can be self-hosted, so sensitive regulatory text embeddings remain within our GCP project, supporting data residency requirements.
- *Negative/Neutral:* PostgreSQL’s vector search performance, while adequate for our prototype (~thousands of documents), might lag behind specialized vector DBs at massive scale (millions of embeddings). Mitigate this via HNSW indexing and by monitoring query latency. If needed, we can scale the database vertically (bigger instances) or consider a migration to a specialized vector store in the future. Also, the footprint of storing large embeddings in Postgres must be managed (ensuring sufficient memory and disk and perhaps partitioning if needed).

Comparison

Feature	pgvector	Weaviate	Pinecone
Open Source	Yes	Yes	No
SQL Support	Excellent	Limited	No
Versioning	Excellent	Moderate	Moderate
Compliance Audit	Excellent	Moderate	Moderate
Metadata Filtering	Excellent	Good	Good
Operational Complexity	Low	Medium	Low

Final Recommendation: PostgreSQL + pgvector

Status: *Accepted* – This decision is implemented in the prototype; regulatory document embeddings are stored in Postgres/pgvector.

ADR 2: Model Hosting and Inference Strategy (Self-Hosted Open-Source LLMs on GCP)

Context: The system relies on Large Language Models (LLMs) for tasks like answering regulatory questions, summarizing documents, and generating compliance assessments. In a regulated financial environment, **data privacy and control are paramount**: user queries and transaction data often contain sensitive information. The design must also comply with the open-source-only requirement of the project. Considered whether to use third-party LLM APIs (which offer state-of-the-art models and easy scaling) versus hosting models ourselves. Key factors included **data security, cost, performance, and development effort**.

Decision: Will **self-host open-source LLMs on our own infrastructure (GCP + Kubernetes)** for all AI inference tasks. Specifically, architecture uses:

- A **small open-source model** for lightweight tasks such as intent classification or summarizing retrieved text (e.g., LLaMA-3.1 8B for classification, which can run on CPU or a single GPU).

- A **larger open-source model** for heavy tasks like multi-regulation Q&A and generating compliance report content (for example, a 13B+ parameter LLaMA-2 model, potentially fine-tuned on instructions).

Deploy these models on a **Kubernetes cluster (GKE)** in our GCP environment. We can leverage a serving framework (such as vLLM or Hugging Face Text Generation Inference) to host the models efficiently. All model inference happens within our secure cloud environment — no data leaves our systems for external processing. This ensures compliance with data governance and meets the assignment’s requirement to avoid proprietary services.

Alternatives Considered:

- *External LLM APIs (OpenAI GPT-4, Azure OpenAI, Google Vertex AI):* These offer top-tier model quality and ease of use (no infra to manage). However, using them would **violate the open-source restriction** and raise data residency concerns. Sending internal compliance queries or transactions to an external service can conflict with regulatory policies and introduces third-party dependency for core functionality. Decided against this approach due to compliance risk and unpredictable recurring costs.
- *On-Premises Proprietary Models:* Explored using licensed enterprise LLMs (e.g., via a vendor that allows on-prem deployment). This could provide strong performance but would incur licensing costs and still involve potential black-box components. It also goes against the open-source prototype directive, so did not pursue this for now.
- *Minimize LLM Usage:* Considered using rule-based logic or smaller ML models for some tasks (e.g., known threshold checks for transaction screening) to reduce reliance on large LLMs. While some straightforward rules are implemented (for example, certain known numeric limits can be checked without an LLM), completely removing LLMs would severely limit the system’s ability to interpret natural language queries or unstructured regulations. We require LLMs to capture the complexity and nuance of regulatory language.

Consequences:

- **Positive: Data Control & Compliance** – The chosen strategy keeps all data processing on our servers, ensuring that sensitive regulatory content and transaction info never leaves our controlled environment. This addresses security and regulatory requirements around data confidentiality. **Flexibility & Customization** – Open-source models allow the team to fine-tune or replace them as needed. There’s no vendor lock-in, and we have full insight into model behavior which aids debugging and ensures reproducibility. **Cost Management** – Running on GCP gives us control to scale hardware up or down and use spot instances or specific GPU types as needed, potentially reducing cost. The architecture remains **model-agnostic**, so switching to a newer or more efficient model in the future is straightforward.
- **Negative/Neutral: Operational Complexity** – Self-hosting large models is more complex than calling an API. We must manage containerization, scaling, and monitoring of model inference services. This added complexity is mitigated by using modern model-serving frameworks and orchestrating via Kubernetes. **Performance & Model Quality** – Open models may be less capable than the latest proprietary models.

We mitigate this by carefully selecting well-regarded models and using retrieval augmentation to improve factual accuracy. Over time, we may consider incorporating more advanced models (including commercial ones) if they can be used in a compliant way. **Infrastructure Cost** – Running large models especially with GPUs can be expensive. We manage this by using smaller models where possible, enabling dynamic scaling (so we only pay for what we use), and possibly using quantized models to reduce resource needs.

Status: *Accepted* – The prototype runs exclusively on self-managed, open-source LLMs within Kubernetes cluster.

ADR 3: Agentic Orchestration Framework & LLM Observability

Context: The Compliance Assistant’s functionality (multi-turn Q&A, transaction analysis with retrieval and reasoning) requires a flexible orchestration layer that can manage a sequence of steps and handle different tools and models. We also need robust **observability** because of strict audit and debugging requirements (every AI-driven decision should have a trace). The question was whether to build a custom orchestration mechanism or leverage an existing framework, and how to ensure thorough logging/tracing of LLM operations without exposing data externally.

Decision: Chosen to use **Google’s Agent Development Kit (ADK)** as the agent orchestration framework, and to integrate an **open-source LLM observability stack (e.g., Langfuse + OpenTelemetry)** for tracing. This means our core application logic uses ADK’s abstractions for building the agent and tools, and we capture detailed logs of each agent step (model inputs/outputs, tool invocations, latencies, errors) for analysis and auditing.

Google ADK is an **open-source, code-first agent framework** tailored for complex multi-step AI workflows. It provides built-in support for tool management, multi-step reasoning, parallel or branchable agent workflows, and solid error-handling patterns. Using ADK enables us to build a reliable agent without starting from scratch, aligning with our need for a **production-quality orchestration layer**.

For observability, we can incorporate **OpenTelemetry** instrumentation in the code and use **Langfuse** (a self-hosted LLM observability platform) to collect and visualize traces. This logs each request’s journey: from initial query to retrieved context to the final output, including intermediate model prompts and decisions. It thus provides the audit trail required by internal auditors.

Alternatives Considered:

- *Custom Orchestration (DIY Agents):* We could manually implement the control flow in plain code. While feasible, this would be time-consuming and error-prone, given the complexity of multi-step flows (especially with potential loops or parallel calls). Instead, adopting ADK allows us to use a well-tested approach with less risk and time investment, plus built-in features like plan optimization.
- *LangChain or Other Agent Frameworks:* We considered other open-source frameworks like LangChain or LlamaIndex, which are popular for RAG applications. These are

viable and have a large community. However, ADK was chosen because it is explicitly designed for building *robust, long-lived agents* with production in mind, and it fits well given our choice of GCP (ADK is an official Google-supported project). This synergy could be valuable for future scalability or integration with GCP's AI ecosystem.

- *Managed Observability Tools (e.g., LangSmith, Arize)*: These services provide advanced monitoring for LLM apps. We opted against them because they are either proprietary or cloud-hosted, which conflicts with keeping all data in-house. They also often involve additional costs. By using Langfuse and standard open-source monitoring stacks (Prometheus/Grafana for metrics, OTel traces), we ensure observability without exposing any data to third parties.

Consequences:

- *Positive: **Maintainable Agent Workflows*** – ADK abstracts away low-level details, making our agent code more maintainable and easier to extend (for example, adding new tools or new agent behaviors like regulatory change analysis). **Reliability** – ADK's patterns for errors/timeouts help maintain a stable user experience, crucial for compliance tasks where an incomplete or inconsistent answer is unacceptable. **Full Traceability** – The chosen observability setup logs everything from initial query to final answer. This provides powerful debugging and model evaluation capabilities (like measuring latency and token usage per step) and meets the audit requirements by persisting the reasoning chain and evidence used for each answer. All logged data stays within our environment, satisfying compliance.
- *Negative/Neutral: **Initial Complexity*** – ADK is a sophisticated framework, so ramping up took some effort. We mitigated this with training and focusing on a subset of features needed for our use cases. **Dependencies** – We are tying our solution to ADK and Langfuse. If either project changes or has issues, we'll need to adapt our solution. Fortunately, both are open-source (with ADK supported by Google, and Langfuse by an active community), reducing the risk of abandonment. **Overhead of Observability** – Capturing detailed LLM traces may slightly increase latency and storage needs. We judge this acceptable given our performance budget and the critical importance of audit logs in compliance. If needed, we can sample or aggregate logs to reduce overhead.

Status: *Accepted* – ADK and open-source observability are implemented in the prototype and will guide future development